

15. 三数之和

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

示例 1：

输入：`nums = [-1,0,1,2,-1,-4]`

输出：`[[-1,-1,2],[-1,0,1]]`

解释：

$nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0$ 。

$nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0$ 。

$nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0$ 。

不同的三元组是 `[-1,0,1]` 和 `[-1,-1,2]`。

注意，输出的顺序和三元组的顺序并不重要。

示例 2：

输入：`nums = [0,1,1]`

输出：`[]`

解释：唯一可能的三元组和不为 0。

示例 3：

输入：`nums = [0,0,0]`

输出：`[[0,0,0]]`

解释：唯一可能的三元组和为 0。

提示

$3 \leq nums.length \leq 3000$

$-105 \leq nums[i] \leq 105$

参考答案

```

class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(),nums.end());
        /*
        对数组进行排序
        这样的话就可以使用滑动窗口这个方法
        */
        vector<vector<int>> res;
        for(int i=0;i<nums.size()-2;i++)
        {
            /*
            这里的i枚举的是第一个数 到 倒数第三个数
            因为一共是三个数
            所以我要在后面留两个数
            不然的话就重复了
            */
            if(i>0 && nums[i]==nums[i-1])
            {
                continue;
            }
            /*
            如果当前i不是第一个元素而且
            当前的数和上一个一样
            说明当前一个循环中的
            和上一个循环都是重复的
            所以就直接跳过当前的循环
            不用枚举当前的数
            */

            /*
            优化一： 如果 nums[i] 与后面最小的两个数相加
            nums[i]+nums[i+1]+nums[i+2]>0,
            那么后面不可能存在三数之和等于 0,
            break 外层循环。
            */
            if(nums[i]+nums[i+1]+nums[i+2]>0)
            {
                break;
            }
            /*
            优化二： 如果 nums[i] 与后面最大的两个数相加
            nums[i]+nums[n-2]+nums[n-1]<0,
            那么内层循环不可能存在三数之和等于 0,
            但继续枚举, nums[i] 可以变大,
            所以后面还有机会找到三数之和等于 0,
            continue 外层循环。
            */
            if(nums[i]+nums[nums.size()-1]+nums[nums.size()-2]<0)
            {
                continue;
            }
        }
    }
}

```

```
int j=i+1;
int k=nums.size()-1;
while(j<k)
{
    if(nums[j]+nums[k]<-nums[i])
    {
        j++;
    }else if(nums[j]+nums[k]>-nums[i])
    {
        k--;
    }else if(nums[j]+nums[k]==-nums[i])
    {
        vector<int> t(3);
        t[0] = nums[i];
        t[1]= nums[j];
        t[2]=nums[k];

        res.push_back(t);

        j++;
        while(j<k &&nums[j]==nums[j-1])
        {
            j++;
        }
        k--;
        while(j<k &&nums[k]==nums[k+1])
        {
            k--;
        }
    }
}
return res;
};
```